

CapTP The Capability Transport Protocol (Spritely Goblins)

Next: [Launching a Tor daemon for Goblins](#), Up: [OCapN: The Object Capabilities Network](#) [[Contents](#)][[Index](#)]

7.1 CapTP: The Capability Transport Protocol

OCapN's CapTP has an abstract “netlayer” interface, allowing communication over many different protocols ranging from Tor Onion Services to IBC to I2P to libp2p to perhaps carrier pigeons with backpacks full of encrypted MicroSD cards. Currently, Goblins only supports the Tor Onion Services netlayer.

Benefits of CapTP:

- Invocation of remote objects resembles the same as with local objects. You can use `<-` and `on` which means that most programs that were originally designed for local-only computing naturally scale out to a networked environment.
- All this is done while mostly hiding the abstraction of the networked protocol from the user.
- Object capability security is upheld. A remote peer cannot make use of any capability that has not been handed to it.
- Live object references between CapTP endpoints are incredibly cheap, merely represented as integers on each side. This keeps message sizes small and efficient.
- CapTP provides (acyclic) distributed garbage collection. Remote peers can indicate when they no longer need an object and the peer locally containing that reference can reclaim it if appropriate.
- Promise pipelining means that messages can be sent to the resolution of a promise before that resolution actually occurs. Over the network this is represented as a pipeline of messages. This can reduce round trips significantly, which is a big win.

To re-quote Mark S. Miller:

Machines grow faster and memories grow larger. But the speed of light is constant and New York is not getting any closer to Tokyo.

- CapTP is written independently of network transport abstractions. It can be run over local Unix domain sockets, an OpenSSL connection, Tor Onion Services, or something custom. CapTP operates under the assumption of secure pairwise channels; the layer which provides the pairwise channels is called “MachineTP” and can be written in a variety of ways. A simple MachineTP abstraction is provided, but presently still requires some manual wiring. In the future, even simpler abstractions will be provided which cover most user needs.

All of these combine into an efficient and powerful protocol that, most powerfully of all, the author of Goblins programs need not think too much about. This circumvents the usual years and years of careful message structure coordination and standardization for many kinds of distributed programs, which can simply be written as normal Goblins programs rather than bespoke protocols.

Limitations of CapTP:

- Goblins' CapTP provides *acyclic* distributed garbage collection. Cycles between servers are not automatically recognized. Full cycle-collecting distributed garbage collection has been written, but requires special cooperation from the garbage collector that is not available in Guile (or in most languages).
- Goblins' CapTP does not do anything about memory usage or resource management on its own. (Features for this will come as Spritely sub-projects in the future.)
- While CapTP in theory routes capabilities to specific remote objects, since the network is mutually suspicious one can't assume that the remote end isn't conspiring some way to hand those capabilities to other unexpected objects. The right way to think about this from an object capability perspective is that a remote misbehaving peer is equivalent to a misbehaving object with the surface area of the entire peer.
- **TEMPORARY:** The API is heavily in flux.

Next: Launching a Tor daemon for Goblins, Up: OCapN: The Object Capabilities Network [Contents][Index]